Sri Lanka Institute of Information Technology



Data Warehousing and Business Intelligence

IT3021

Year 3, Semester 1, 2026

Assignment 01

Campus : Malabe

Name : K.N.Dilhara

ID : IT23842526

Email : IT23842526@my.sliit.lk

Contact : +94 71 2149 760

Table of Contents

1.Data Set Selection

1.1 Introduction
1.2 Dataset Overview
1.3 Source Types and Formats
1.4 Coverage and Realism
1.5 Suitability for Data Warehousing and Business Intelligence
1.6 Entity–Relationship Diagram of the Source Dataset

2. Source Data Description

2.1 CSV Source
2.2 Text File Source
2.3 Data Integration Strategy
2.4 Data Quality Considerations
2.5 Summary

3. Solution Architecture

3.1 Architectural Diagram
3.2 Component Summary
3.3 Benefits of the Architecture

4. Data Warehouse Design & Development

4.1 Dimensional Model Description
4.2 Relational Diagram
4.3 Design Assumptions
4.4 Key Design Benefits

5. ETL Development

5.1 SSIS Packages Overview
5.2 Connection Managers
5.3 Package 1: Accident_Load_Staging.dtsx
5.4 Package 2: Accident_Load_DW.dtsx
5.4a Data Flow – Load DimDate
5.4b Data Flow – Load DimLocation (SCD Type 2)
5.4c Data Flow – Load FactAccident
5.5 Package 3: UpdateFactAccident.dtsx
5.6 Data Validation and Results
5.5 Package 3: UpdateFactEnrollment.dtsx

6. Conclusion

1. Data Set Selection

1.1 Introduction

For this assignment, a real-world accident dataset was selected as the OLTP-style data source. The dataset represents road accident records and includes detailed information about crash events such as date, time, location, and injury statistics.

This dataset was chosen because it simulates a realistic operational system where accident records are continuously generated. It does not contain any pre-built fact or dimension structures, making it suitable for designing a data warehouse from scratch.

Additionally, the dataset is not used in practical sessions and is not based on the AdventureWorks database, ensuring compliance with assignment requirements.

1.2 Dataset Overview

The selected dataset contains accident records with multiple attributes describing each crash event. The key attributes include:

- **CRASH DATE** – Date of the accident
- **CRASH TIME** – Time of the accident
- **BOROUGH** – Area where the accident occurred
- **ZIP CODE** – Postal code of the location
- **LATITUDE & LONGITUDE** – Geographic coordinates
- **ON STREET NAME, CROSS STREET NAME, OFF STREET NAME** – Location details
- **NUMBER OF PERSONS INJURED / KILLED** – Overall accident impact
- **NUMBER OF PEDESTRIANS INJURED / KILLED** – Pedestrian-related impact
- **NUMBER OF CYCLIST INJURED / KILLED** – Cyclist-related impact
- **NUMBER OF MOTORIST INJURED / KILLED** – Motorist-related impact
- **CONTRIBUTING FACTOR VEHICLE (1–5)** – Causes of accidents
- **VEHICLE TYPE CODE (1–5)** – Types of vehicles involved
- **COLLISION_ID** – Unique identifier for each accident

The dataset represents a detailed and multi-dimensional view of accident events, enabling comprehensive analysis across time, location, and severity.

1.3 Source Types and Formats

To simulate a realistic data integration scenario, the dataset was divided into multiple source types:

1. CSV Source (Accident Data)

- File: accidents_main.csv
- Contains:
    - Collision ID
    - Crash Date and Time
    - Injury and death statistics
    - Vehicle types and contributing factors

2. Text File Source (Location Data)

- File: location_data.txt
- Contains:
    - Collision ID
    - Borough
    - ZIP Code
    - Latitude and Longitude
    - Street names

This separation ensures the presence of **multiple** heterogeneous data sources, which is essential for demonstrating ETL capabilities.

1.4 Coverage and Realism

The dataset contains accident records covering approximately one year of data. It includes a wide range of real-world scenarios such as:

- Accidents occurring at different times of the day
- Variations across different boroughs and locations
- Multiple categories of injuries and fatalities
- Different vehicle types and contributing factors

This diversity makes the dataset highly realistic and suitable for analytical processing.

1.5 Suitability for Data Warehousing and Business Intelligence

The selected dataset is highly suitable for data warehousing and business intelligence for the following reasons:

Requirement	**Explanation**
OLTP-style dataset	Contains raw transactional accident records
Sufficient data volume	Includes multiple records across time and locations
Multiple data sources	CSV and text files used
Supports ETL operations	Requires joins, transformations, and lookups
Supports dimensional modeling	Can be structured into fact and dimension tables
Enables BI analysis	Supports trend analysis, location analysis, and severity analysis

1.6 Entity–Relationship Diagram of the Source Dataset

The source dataset can be conceptually represented using an Entity–Relationship (ER) model.

Main Entities:

- **Accident**
- **Location**
- **Vehicle**

- **Contributing Factor**

Relationships:

- Each accident is associated with one location
- Each accident may involve multiple vehicles
- Each accident may have multiple contributing factors

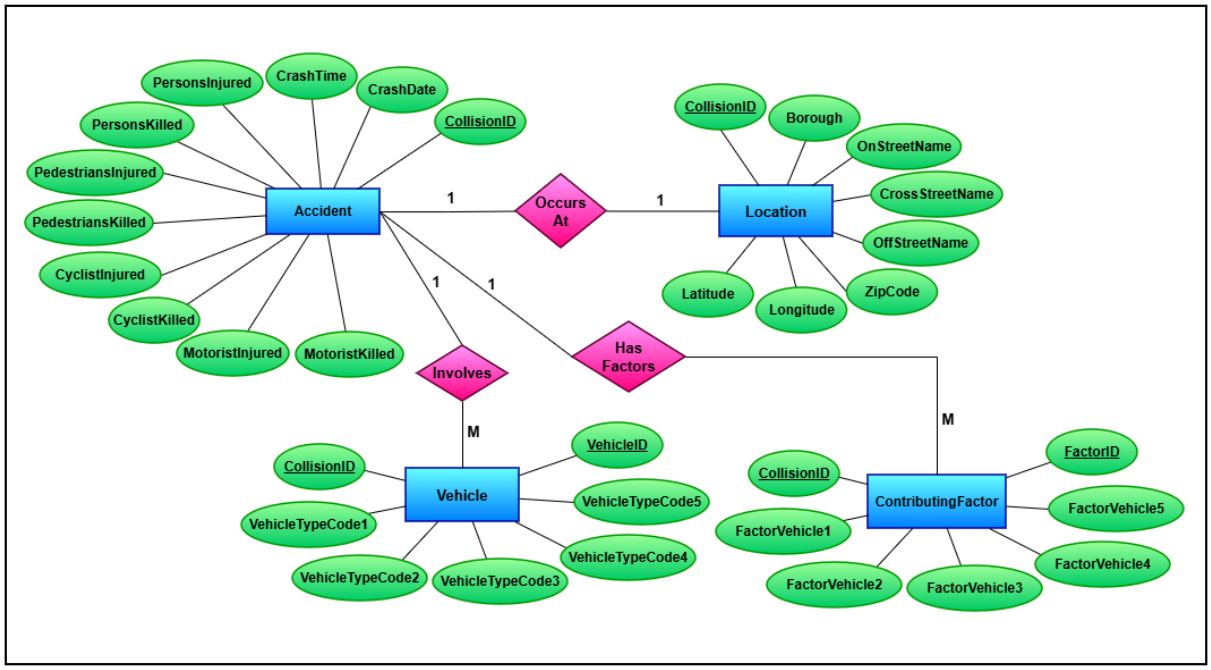


Figure 1: Entity–Relationship Diagram of the Accident Source Dataset

2. Source Data Description

2.1 CSV Source

The primary data source used in this project is a Comma-Separated Values (CSV) file named accidents_main.csv. This file contains the core accident-related transactional data and represents the main OLTP-style dataset.

Structure of CSV Source

The CSV file includes the following key attributes:

- **COLLISION_ID** – Unique identifier for each accident

- **CRASH DATE** – Date on which the accident occurred
- **CRASH TIME** – Time of the accident
- **NUMBER OF PERSONS INJURED** – Total number of people injured
- **NUMBER OF PERSONS KILLED** – Total number of fatalities
- **NUMBER OF PEDESTRIANS INJURED / KILLED** – Pedestrian-related impact
- **NUMBER OF CYCLIST INJURED / KILLED** – Cyclist-related impact
- **NUMBER OF MOTORIST INJURED / KILLED** – Motorist-related impact
- **VEHICLE TYPE CODE (1–5)** – Types of vehicles involved in the accident
- **CONTRIBUTING FACTOR VEHICLE (1–5)** – Causes contributing to the accident

Purpose of CSV Source

The CSV file serves as the primary source of measurable data (facts). It provides the numerical values required for analysis, such as injury counts and fatality statistics.

This data is loaded into the staging table Stg_Accident and later transformed into the **FactAccident table** in the data warehouse.

2.2 Text File Source

A secondary data source was created in the form of a text file named location_data.txt. This file contains location-related information separated from the main dataset to simulate a multi-source environment.

Structure of Text File Source

The text file includes the following attributes:

- **COLLISION_ID** – Used to link with accident data
- **BOROUGH** – Area where the accident occurred
- **ZIP CODE** – Postal code of the location
- **LATITUDE** – Geographic coordinate
- **LONGITUDE** – Geographic coordinate
- **ON STREET NAME** – Street where accident occurred

- **CROSS STREET NAME** – Nearby intersecting street
- **OFF STREET NAME** – Additional street reference

Purpose of Text File Source

The text file serves as the source for location-related attributes, which are used to populate the DimLocation dimension table.

By separating location data into a different source:

- It demonstrates the integration of **heterogeneous data sources**
- It improves clarity in data modeling
- It enables the implementation of Slowly Changing Dimensions (SCD Type 2)

2.3 Data Integration Strategy

To combine data from both sources, the COLLISION_ID field is used as the common key.

Integration Process:

- Data from accidents_main.csv is loaded into Stg_Accident
- Data from location_data.txt is loaded into Stg_Location
- Both tables are joined using:

Stg_Accident.COLLISION_ID = Stg_Location.COLLISION_ID

This integration ensures that accident records are enriched with corresponding location details before loading into the data warehouse.

2.4 Data Quality Considerations

During the data preparation stage, several data quality issues were addressed:

- **Handling NULL values** in location and injury columns
- **Standardizing data types** (e.g., date, numeric values)
- **Removing duplicate records** using DISTINCT queries
- **Ensuring referential integrity** between accident and location data

These steps ensured that clean and reliable data is used for ETL processing and data warehouse loading.

2.5 Summary

The use of both CSV and text file sources provides a realistic data integration scenario. The separation of accident and location data enables effective dimensional modeling and demonstrates the ability to handle multiple data sources in a data warehousing environment.

3. Solution Architecture

3.1 Architectural Diagram

A high-level Data Warehouse (DW) and Business Intelligence (BI) architecture was designed to support the integration, transformation, and analysis of accident data.

The architecture consists of multiple layers, starting from raw data sources and ending with the structured data warehouse for analytical processing.

Architecture Flow

Description of Data Flow

1. **Source Layer**
   The system uses two types of data sources:
   - A CSV file containing accident-related data
   - A text file containing location data
2. **Staging Layer**
   Data from both sources is first loaded into staging tables:
   - Stg_Accident
   - Stg_Location

This layer is used to clean, validate, and prepare data before transformation.

3. **ETL Layer (SSIS)**
   SQL Server Integration Services (SSIS) is used to:
   - Extract data from staging tables
   - Transform data using lookups, derived columns, and joins
   - Load data into dimension and fact tables
4. **Data Warehouse Layer**
   The final data is stored in a structured format using a **star schema**, consisting of:
   - DimDate
   - DimLocation
   - FactAccident

This architecture ensures efficient data processing and supports analytical queries.

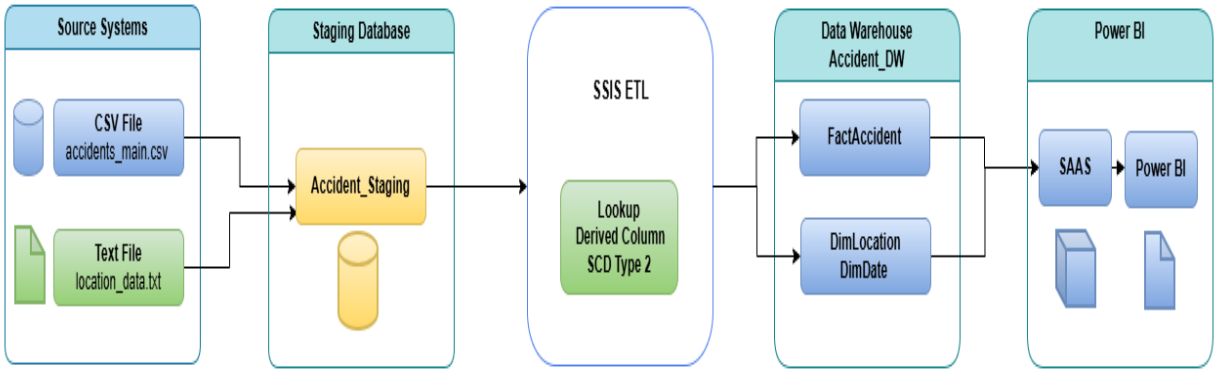


Figure 2: High-Level DW & BI Architecture for Accident Dat

3.2 Component Summary

The following table summarizes the main components of the solution:

Layer	Component	Description
Source Systems	accidents_main.csv	Contains accident transaction data including injuries, fatalities, and crash details

Layer	Component	Description
Source Systems	location_data.txt	Contains location attributes such as borough, ZIP code, and street names
Staging Area	Accident_Staging	Temporary storage used to clean and prepare raw data before loading
ETL Process	SSIS (SQL Server Integration Services)	Performs data extraction, transformation, and loading operations
Data Warehouse	Accident_DW	Final structured database using star schema
Dimension Tables	DimDate, DimLocation	Stores descriptive attributes for analysis
Fact Table	FactAccident	Stores measurable accident data
BI Layer	Future Integration (SSAS / Power BI)	Can be used for reporting and analytical dashboards

3.3 Benefits of the Architecture

The designed architecture provides several advantages:

- **Data Integration**
  Combines data from multiple sources (CSV and text file)
- **Improved Data Quality**
  Staging layer ensures data is cleaned before loading
- **Scalability**
  Additional data sources can be added easily
- **Efficient Query Performance**
  Star schema improves analytical query performance
- **Support for Business Intelligence**
  Enables future reporting using tools such as Power BI

4. Data Warehouse Design & Development

4.1 Dimensional Model Description

A **star schema** was designed for the accident data warehouse to support efficient analytical processing. The model consists of one central fact table and multiple dimension tables.

The schema captures accident-related measures and connects them with descriptive attributes such as date and location.

Tables in the Data Warehouse

Table Name	Type	Description
FactAccident	Fact Table	Stores accident-related measures such as injuries and fatalities
DimDate	Dimension	Stores date attributes for time-based analysis
DimLocation	Dimension	Stores location information and supports historical tracking using SCD Type 2

Fact Table – FactAccident

The FactAccident table is the central table of the data warehouse and contains measurable data.

Key Attributes:

- DateKey (Foreign Key → DimDate)
- LocationKey (Foreign Key → DimLocation)
- PersonsInjured
- PersonsKilled
- PedestriansInjured
- CyclistsInjured
- MotoristsInjured

This table allows analysis of accident severity across different dimensions.

Dimension Table – DimDate

The DimDate table provides time-related attributes.

Attributes:

- DateKey (Primary Key)
- FullDate
- Day
- Month
- Year
- Quarter

Purpose:

- Enables time-based analysis such as trends by day, month, or year
- Supports aggregation and grouping

Dimension Table – DimLocation (SCD Type 2)

The DimLocation table stores accident location details and is implemented as a **Slowly Changing Dimension Type 2**.

Attributes:

- LocationKey (Surrogate Key)
- Borough
- ZipCode
- Latitude
- Longitude
- OnStreetName
- CrossStreetName

- OffStreetName
- StartDate
- EndDate
- IsCurrent

SCD Type 2 Implementation

The SCD Type 2 approach is used to track historical changes in location data.

Behavior:

- When a change occurs, a new record is inserted
- The previous record is updated with an EndDate
- The current record is marked using IsCurrent = 1

Example:

LocationKey	Borough	ZipCode	StartDate	EndDate	IsCurrent
1	Bronx	10451	2020-01-01	NULL	1

This ensures that historical location data is preserved for accurate analysis.

4.2 Relational Diagram

The dimensional model follows a star schema, where the fact table is connected to dimension tables using foreign keys.

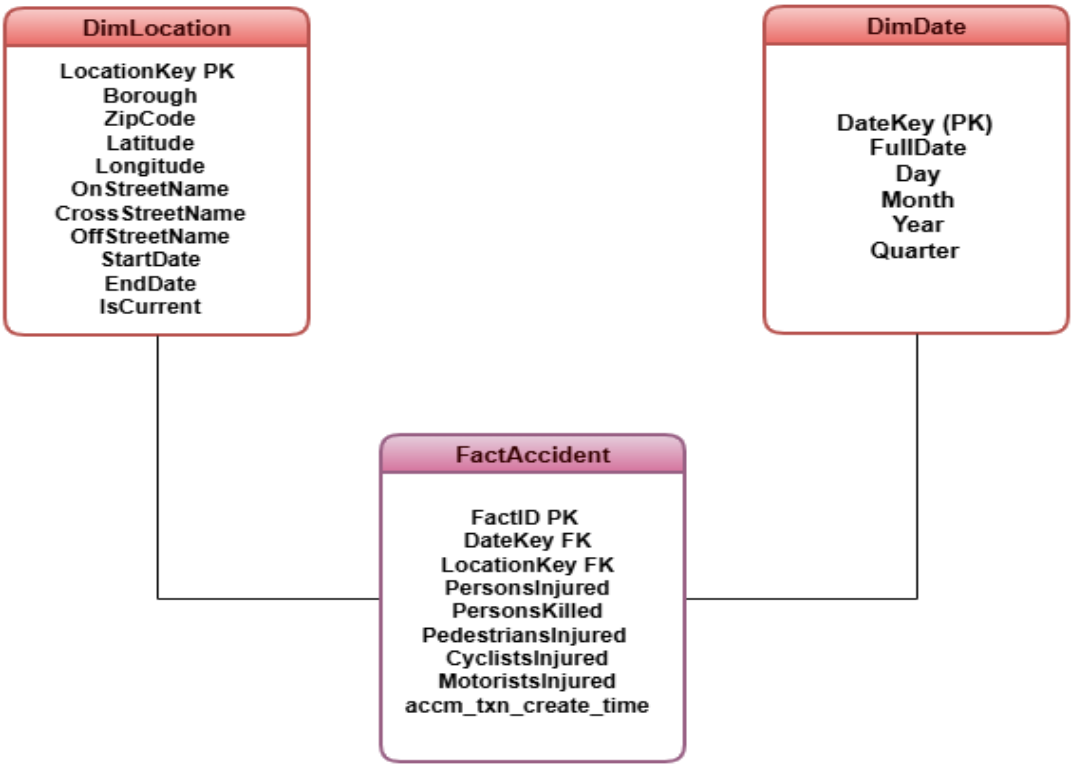


Figure 3: Star Schema of Accident Data Warehouse

4.3 Design Assumptions

The following assumptions were made during the design of the data warehouse:

- A **star schema** was chosen due to its simplicity and efficiency for analytical queries
- Each record in the **FactAccident** table represents a single accident event
- The **DimDate** table is used as a conformed dimension for time-based analysis
- The **DimLocation** table uses SCD Type 2 to maintain historical changes
- Surrogate keys (DateKey and LocationKey) are used instead of natural keys
- The dataset is assumed to contain consistent and valid relationships between accident and location data
- Derived attributes such as transaction processing time are calculated during ETL

4.4 Key Design Benefits

The designed data warehouse provides several advantages:

- **Efficient querying** using star schema structure
- **Improved performance** for aggregation and reporting
- **Historical tracking** using SCD Type 2
- **Scalability** to add more dimensions (e.g., Vehicle, Contributing Factor)
- **Support for BI tools** such as Power BI and SSAS

5. ETL Development

The Extract, Transform, Load (ETL) process was implemented using SQL Server Integration Services (SSIS) to move data from source systems into the data warehouse. The ETL workflow ensures that data is properly extracted, cleaned, transformed, and loaded into dimension and fact tables.

The ETL implementation was divided into multiple packages to ensure modularity and maintainability.

5.1 SSIS Packages Overview

Three SSIS packages were developed to handle different stages of the ETL process:

Package Name	Purpose
Accident_Load_Staging.dtsx	Extracts data from CSV and text file sources into staging tables
Accident_Load_DW.dtsx	Transforms and loads data into dimension and fact tables
UpdateFactAccident.dtsx	Updates accumulating fact table attributes such as completion time

5.2 Connection Managers

Several connection managers were configured to handle data integration:

OLE DB Connections

- **Accident_Staging** – Used to store raw data in staging tables
- **Accident_DW** – Used for loading data into the data warehouse

Flat File Connections

- **accidents_main.csv** – Contains accident transaction data
- **location_data.txt** – Contains location information

These connection managers allow SSIS to connect and transfer data between different sources.

5.3 Package 1: Accident_Load_Staging.dtsx

This package is responsible for extracting data from source files and loading it into staging tables.

Control Flow Tasks:

1. Load Accident Data
2. Load Location Data

Data Flow – Load Accident Data

- **Flat File Source** reads data from accidents_main.csv
- Data is directly loaded into Stg_Accident using **OLE DB Destination**

Data Flow – Load Location Data

- **Flat File Source** reads data from location_data.txt
- Data is loaded into Stg_Location

Evidence of Row Counts: Data Load into Staging Tables

These figures show sample records from the staging tables, confirming that raw data has been successfully loaded from source files into the staging database.

```
1  SELECT TOP 10 * FROM Stg_Accident;
2
```

110 % No issues found

Results Messages

	CollisionID	CrashDate	CrashTime	PersonsInjured	PersonsKilled	PedestriansInjured	PedestriansKilled	CyclistInjured	CyclistKilled	MotoristInjured	MotoristKilled
1	4342908	2020-08-29	15:40:00.0000000	0	0	0	0	0	0	0	0
2	4343555	2020-08-29	21:00:00.0000000	2	0	0	0	0	0	2	0
3	4343142	2020-08-29	18:20:00.0000000	1	0	1	0	0	0	0	0
4	4343588	2020-08-29	00:00:00.0000000	0	0	0	0	0	0	0	0
5	4342953	2020-08-29	17:10:00.0000000	0	0	0	0	0	0	0	0
6	4342721	2020-08-29	03:29:00.0000000	1	0	0	0	0	0	1	0
7	4343004	2020-08-29	19:30:00.0000000	0	0	0	0	0	0	0	0
8	4343342	2020-08-29	00:00:00.0000000	0	0	0	0	0	0	0	0
9	4343030	2020-08-29	19:50:00.0000000	0	0	0	0	0	0	0	0
10	4343040	2020-08-29	09:20:00.0000000	0	0	0	0	0	0	0	0

Query executed successfully. DESKTOP-C4J2A1E\MSSQLSERVER... sa (53) Accid

Figure 4: Sample Data from Stg_Accident Table

```
1
2  SELECT TOP 10 * FROM Stg_Location;
```

110 % No issues found Ln: 2, Ch:

Results Messages

	CollisionID	CrashDate	CrashTime	PersonsInjured	PersonsKilled	PedestriansInjured	PedestriansKilled	CyclistInjured	CyclistKilled	MotoristInjured	MotoristKilled
1	4342908	2020-08-29	15:40:00.0000000	0	0	0	0	0	0	0	0
2	4343555	2020-08-29	21:00:00.0000000	2	0	0	0	0	0	2	0
3	4343142	2020-08-29	18:20:00.0000000	1	0	1	0	0	0	0	0
4	4343588	2020-08-29	00:00:00.0000000	0	0	0	0	0	0	0	0
5	4342953	2020-08-29	17:10:00.0000000	0	0	0	0	0	0	0	0
6	4342721	2020-08-29	03:29:00.0000000	1	0	0	0	0	0	1	0
7	4343004	2020-08-29	19:30:00.0000000	0	0	0	0	0	0	0	0
8	4343342	2020-08-29	00:00:00.0000000	0	0	0	0	0	0	0	0
9	4343030	2020-08-29	19:50:00.0000000	0	0	0	0	0	0	0	0
10	4343040	2020-08-29	09:20:00.0000000	0	0	0	0	0	0	0	0

Query executed successfully. DESKTOP-C4J2A1E\MSSQLSERVER... sa (53) Ac

Figure 5: Sample Data from Stg_Location Table

5.4 Package 2: Accident_Load_DW.dtsx

This package handles transformation and loading of data into the data warehouse.

Execution Order:

1. Load DimDate
2. Load DimLocation
3. Load FactAccident

This order ensures that dimension tables are populated before the fact table.

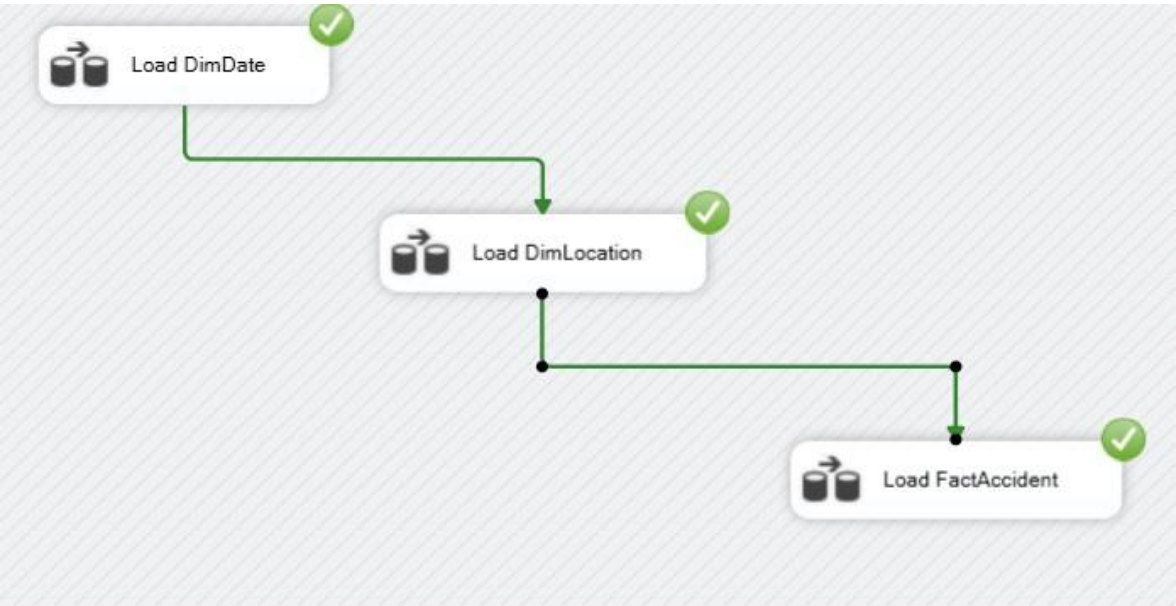


Figure 6: Control Flow of Load_DW.dtsx

5.4a Data Flow – Load DimDate

This data flow extracts distinct dates from staging and loads them into the DimDate table.

Steps:

- **OLE DB Source** reads crash dates from Stg_Accident
- **Derived Column Transformation** creates:
    - DateKey
    - Day

    - Month
    - Year
    - Quarter
- Data is loaded into DimDate using **OLE DB Destination**

5.4b Data Flow – Load DimLocation (SCD Type 2)

This data flow loads location data and implements Slowly Changing Dimension Type 2.

Steps:

- **OLE DB Source** reads data from Stg_Location
- **Slowly Changing Dimension (SCD) Transformation**:
    - Detects new and changed records
    - Updates old records (EndDate)
    - Inserts new records (StartDate)
- Data is loaded into DimLocation

This ensures historical tracking of location changes.

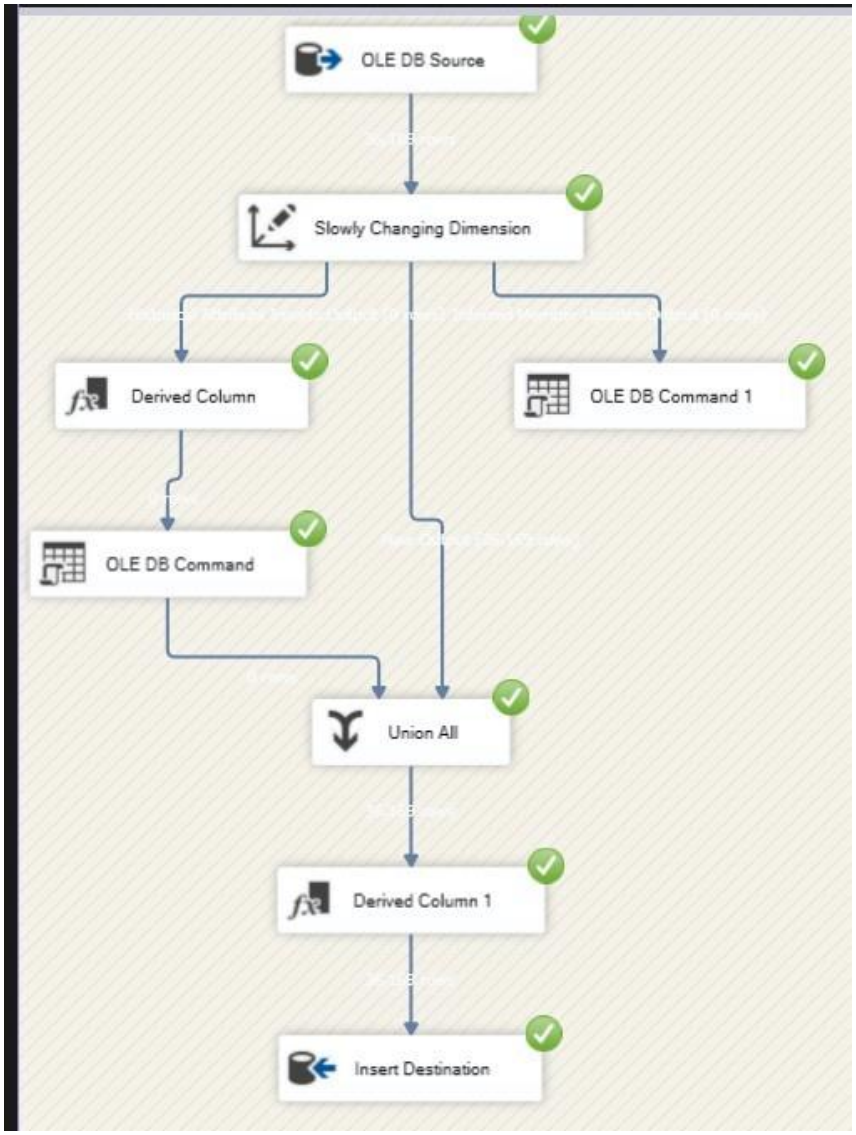


Figure 7: Data Flow for Loading DimLocation with SCD Type 2

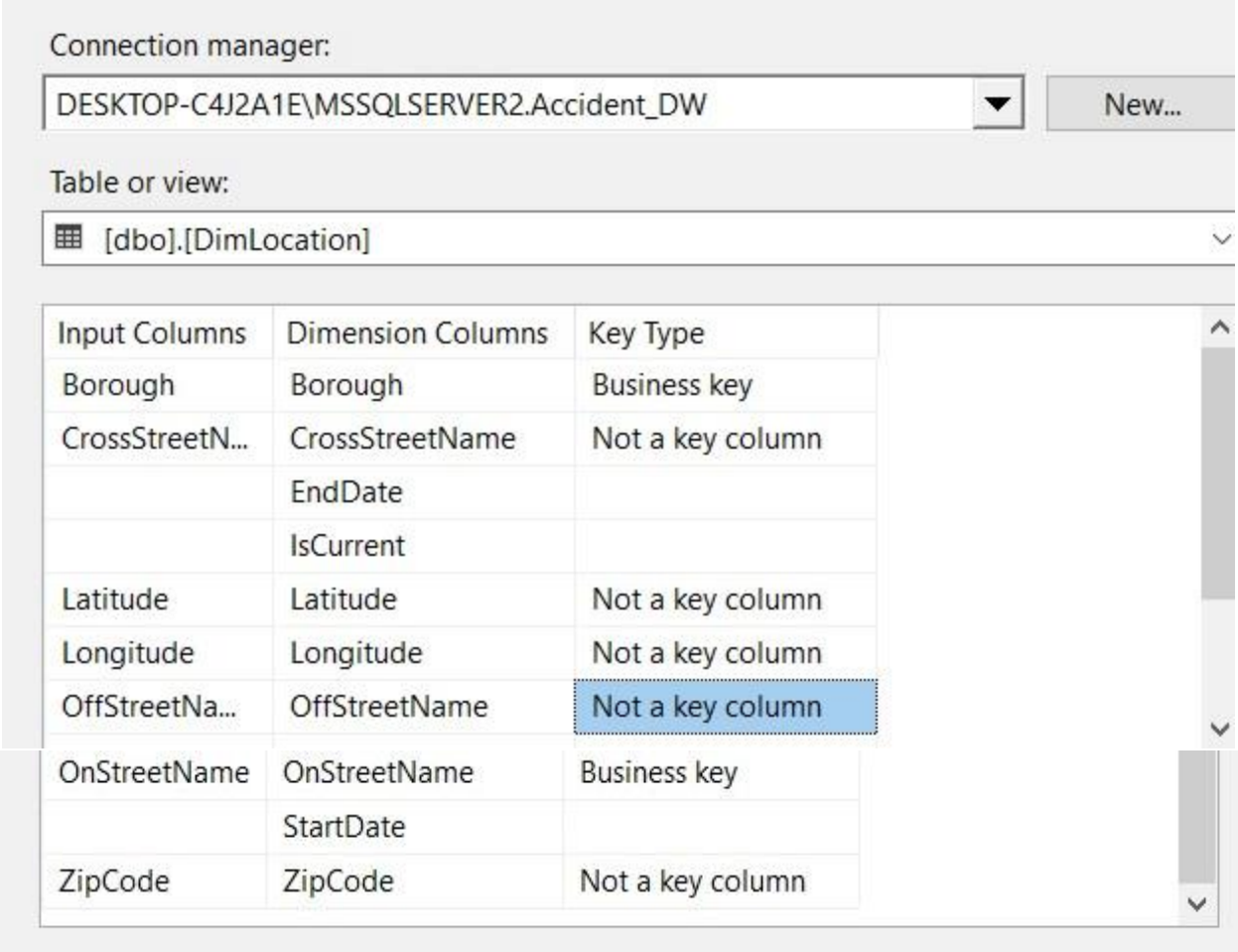


Figure 8: SCD Column Mapping Configuration

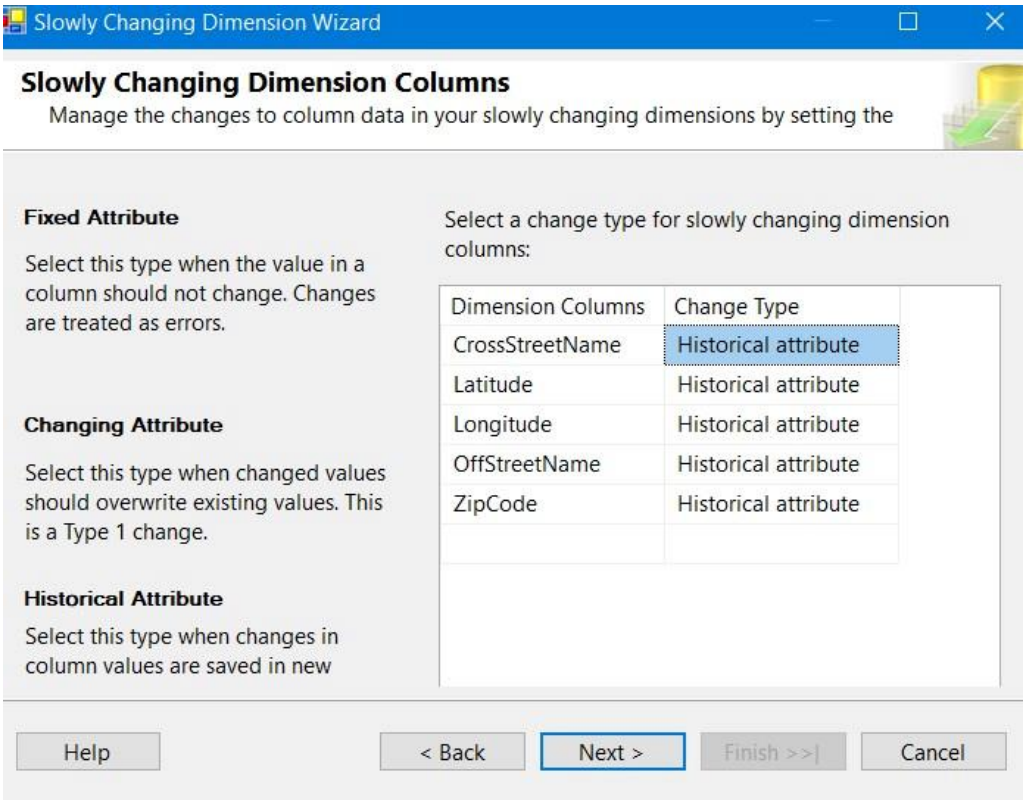


Figure 9: SCD Type 2 Attribute Configuration

5.4c Data Flow – Load FactAccident

This is the main data flow that loads the fact table.

Steps:

- **OLE DB Source** joins:
    - Stg_Accident
    - Stg_Location
      using COLLISION_ID
- **Lookup Transformations**:
    - Retrieve **DateKey** from DimDate
    - Retrieve **LocationKey** from DimLocation
- **Derived Column Transformation**:
    - Adds accm_txn_create_time using GETDATE()
- **OLE DB Destination**:

- Loads data into FactAccident

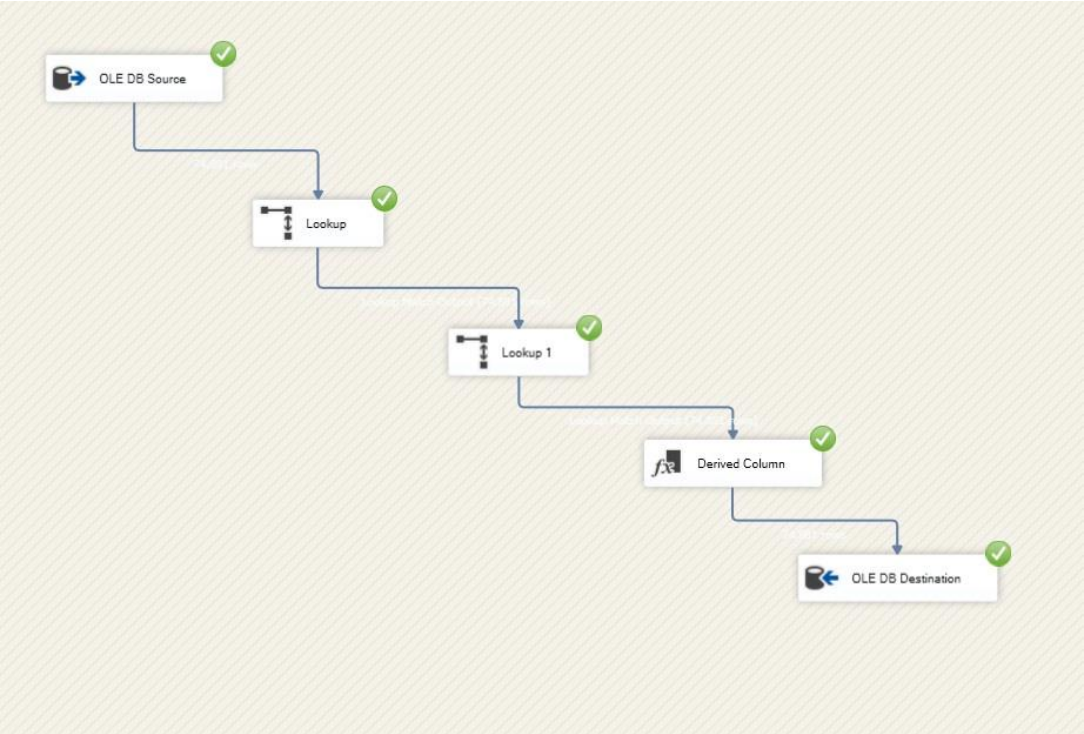


Figure 10: Data Flow for Loading FactAccident

5.5 Package 3: UpdateFactAccident.dtsx

This package demonstrates the use of an **accumulating fact table**.

Process Overview:

1. **Stage Completion Data**
    - Completion timestamps are loaded from a separate dataset (CSV or table)
2. **Update Fact Table**
    - SQL query updates:
        - accm_txn_complete_time
        - txn_process_time_hours
3. **Clean Staging Table**
    - Temporary data is removed after processing

Fact Table Update Logic

The following SQL logic is used:

```
UPDATE FactAccident
SET accm_txn_complete_time = GETDATE();

UPDATE FactAccident
SET txn_process_time_hours =
   DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time);
```

Purpose

This process enables tracking of:

- Data processing time
- ETL performance
- Transaction lifecycle

5.6 Data Validation and Results

After executing the ETL packages, the following validations were performed:

- Verified row counts in staging tables
- Verified data in dimension tables
- Verified data in fact table
- Ensured foreign key relationships are correct

Evidence of Data Load into Dimension Tables

```
1   SELECT TOP 10 * FROM DimDate;
```

110 % No issues found

Results Messages

	DateKey	FullDate	Day	Month	Year	Quarter
1	20200101	2020-01-01	1	1	2020	1
2	20200102	2020-01-02	2	1	2020	1
3	20200103	2020-01-03	3	1	2020	1
4	20200104	2020-01-04	4	1	2020	1
5	20200105	2020-01-05	5	1	2020	1
6	20200106	2020-01-06	6	1	2020	1
7	20200107	2020-01-07	7	1	2020	1
8	20200108	2020-01-08	8	1	2020	1
9	20200109	2020-01-09	9	1	2020	1
10	20200110	2020-01-10	10	1	2020	1

Click to select the whole column

Figure 11: Data Loaded into DimDate Table

```
1   SELECT * FROM DimLocation;
```

110 % No issues found

Results Messages

	LocationKey	Borough	ZipCode	Latitude	Longitude	OnStreetName	CrossStreetName	OffStreetName	StartDate	EndDate	IsCurrent
25...	25107	BROO...	11226	40.65...	-73.96527	EAST 16 STREET	CATON AVENUE	NULL	NULL	NULL	0
25...	25108	BROO...	11226	40.65...	-73.95296	CHURCH AVENUE	VERONICA PLACE	NULL	NULL	NULL	0
25...	25109	BROO...	11226	40.65...	-73.95248	CHURCH AVENUE	ROGERS AVENUE	NULL	NULL	NULL	0
25...	25110	BROO...	11226	40.65...	-73.95248	ROGERS AVENUE	CHURCH AVENUE	NULL	NULL	NULL	0
25...	25111	BROO...	11226	40.65...	-73.951...	LLOYD STREET	CHURCH AVENUE	NULL	NULL	NULL	1
25...	25112	BROO...	11226	40.65...	-73.951...	NULL	NULL	2812 CHURCH AVENUE	NULL	NULL	1
25...	25113	BROO...	11226	40.65...	-73.94958	CHURCH AVENUE	NOSTRAND AVENUE	NULL	NULL	NULL	0
25...	25114	BROO...	11226	40.65...	-73.94958	NOSTRAND AVEN...	CHURCH AVENUE	NULL	NULL	NULL	0
25...	25115	BROO...	11226	40.65...	-73.9493	NULL	NULL	3018 CHURCH AVENUE	NULL	NULL	1
25...	25116	BROO...	11226	40.65...	-73.94943	NULL	NULL	3001 CHURCH AVENUE	NULL	NULL	1
25...	25117	BROO...	11226	40.65...	-73.961...	NULL	NULL	20 SAINT PAULS COU...	NULL	NULL	1

Query executed successfully. DESKTOP-C4J2A1F\MSSQLSERVER... sa (60)

Figure 12: Data Loaded into DimLocation Table

```
1  SELECT TOP 10 * FROM FactAccident;
```

110 % | No issues found | Ln: 1, Ch: 35 | (34 chars, 1 lines) | TABS | CR

Results | Messages

	FactID	DateKey	LocationKey	PersonsInjured	PersonsKilled	PedestriansInjured	CyclistsInjured	MotoristsInjured	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	1	20200729	6239	0	0	0	0	NULL	2026-03-23 23:06:15.933	2026-03-23 23:26:40.647	0.333333
2	2	20200729	34219	1	0	0	1	NULL	2026-03-23 23:06:15.933	2026-03-23 23:26:40.647	0.333333
3	3	20200729	25464	1	0	0	0	NULL	2026-03-23 23:06:15.933	2026-03-23 23:26:40.647	0.333333
4	4	20200729	41371	0	0	0	0	NULL	2026-03-23 23:06:15.933	2026-03-23 23:26:40.647	0.333333
5	5	20200729	34219	1	0	0	0	NULL	2026-03-23 23:06:15.933	2026-03-23 23:26:40.647	0.333333
6	6	20200729	26875	0	0	0	0	NULL	2026-03-23 23:06:15.933	2026-03-23 23:26:40.647	0.333333
7	7	20200729	34104	0	0	0	0	NULL	2026-03-23 23:06:15.933	2026-03-23 23:26:40.647	0.333333
8	8	20200729	34219	0	0	0	0	NULL	2026-03-23 23:06:15.933	2026-03-23 23:26:40.647	0.333333
9	9	20200729	34219	0	0	0	0	NULL	2026-03-23 23:06:15.933	2026-03-23 23:26:40.647	0.333333
10	10	20200729	5075	0	0	0	0	NULL	2026-03-23 23:06:15.933	2026-03-23 23:26:40.647	0.333333

Query executed successfully. | DESKTOP-C4J2A1E\MSSQLSERVER... | sa (60) | Accident_DW | 00:00:00 | Row: 1

Figure 13: Data Loaded into FactAccident Table

6. Conclusion

This assignment successfully demonstrated the design and implementation of a complete Data Warehouse solution for accident data analysis using Microsoft SQL Server and SQL Server Integration Services (SSIS).

The project involved integrating data from multiple heterogeneous sources, including a CSV file containing accident data and a text file containing location details. These sources were extracted and loaded into staging tables, where data cleaning and transformation processes were performed to ensure accuracy and consistency.

A star schema-based dimensional model was designed, consisting of one central fact table (FactAccident) and multiple dimension tables (DimDate and DimLocation). This structure enables efficient analytical querying and supports various business intelligence operations such as trend analysis, aggregation, and reporting.

Key data warehousing concepts were successfully implemented, including:

- **ETL Process** using SSIS for data extraction, transformation, and loading
- **Slowly Changing Dimension (Type 2)** to track historical changes in location data
- **Lookup transformations** for resolving surrogate keys
- **Derived columns** for calculating additional attributes

- **Accumulating fact table logic** to track transaction processing time

The modular design of ETL packages improved maintainability and ensured a logical separation between staging, transformation, and data warehouse loading processes.

Although advanced reporting tools such as SSAS and Power BI were not implemented in this assignment, the designed data warehouse is fully compatible with these tools and can be extended for future analytical and reporting purposes.

Overall, this project provided valuable hands-on experience in data integration, dimensional modeling, ETL development, and data quality management. It highlights the importance of data warehousing in transforming raw operational data into meaningful insights that support decision-making.